

Sistem Programlama: Vize/Final Hazırlık Rehberi

Dosya Sistemleri, Tamponlama (Buffering) ve Kabuk
Yönlendirmeleri Üzerine Kapsamlı Sınav Özeti

Sınav Yol Haritası

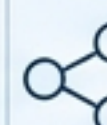
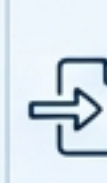


1. G/Ç ve Tamponlama

-  - Sistem Çağrıları
-  - Standart Kütüphane
-  - Performans Optimizasyonları
-  - simpcat Analizi

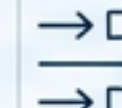
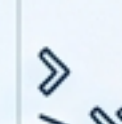


2. Dosya Anatomisi

-  - İnode Yapısı ve Meta Veri
-  - Veri Blokları
-  - Hard Link Dinamikleri
-  - Soft (Symbolic) Link Farkları



3. Kabuk Yönlendirmesi

-  - Standart Akışlar (0, 1, 2)
-  - Yönlendirme Operatörleri (>, <, >>)
-  - 2>&1 İşlem Sırası Tuzakları

Sistem Çağrıları vs. Standart G/Ç Kütüphanesi

Sistem Çağrıları (System Calls)

open, close, read, write, lseek

- **Mekanizma:** İşletim sistemine doğrudan istek yapar. Her çağrıda CPU devralınır ve bağlam değişimi (*context switch*) gerekir. İşlem yavaştır.
- **Tanımlayıcı:** Tamsayı dosya tanımlayıcıları (integer file descriptors - örn. 0, 1, 2) kullanır.

Standart G/Ç Çağrıları (Standard I/O)

fopen, fclose, fread, fwrite, getchar, printf

- **Mekanizma:** İşletim sistemi yerine C standart kütüphanesine gider. Arka planda otomatik tamponlama (buffering) yapar. İşlem çok hızlıdır.
- **Tanımlayıcı:** **FILE** adı verilen bir yapı (struct) ve işaretçiler (pointers) ile çalışır.

Sınav Notu

Hız ve optimizasyon sorulursa: Standart G/Ç (tamponlanmış) her zaman tekil Sistem Çağrılarından daha yüksek performans gösterir.

Performans Felaketi: Neden Tamponlama Şart?

simpcat1.c (getchar / putchar)

```
#include <stdio.h>
int main()
{
    char c;
    c = getchar();
    while (c != EOF) {
        putchar(c);
        c = getchar();
    }
    return 0;
}
```



1.6s
Standart G/Ç
(Tamponlanmış)

simpcat2.c (read / write)

```
#include <unistd.h>
int main()
{
    char c;
    while(read(0, &c, 1) == 1) {
        write(1, &c, 1);
    }
    return 0;
}
```



23.0s
Sistem Çağrısı
(Tamponsuz)

simpcat3.c (fread / fwrite)

```
#include <stdio.h>
int main()
{
    char c;
    i = fread(&c, 1, 1, stdin);
    while (i > 0) {
        fwrite(&c, 1, 1, stdout);
        i = fread(&c, 1, 1, stdin);
    }
    return 0;
}
```

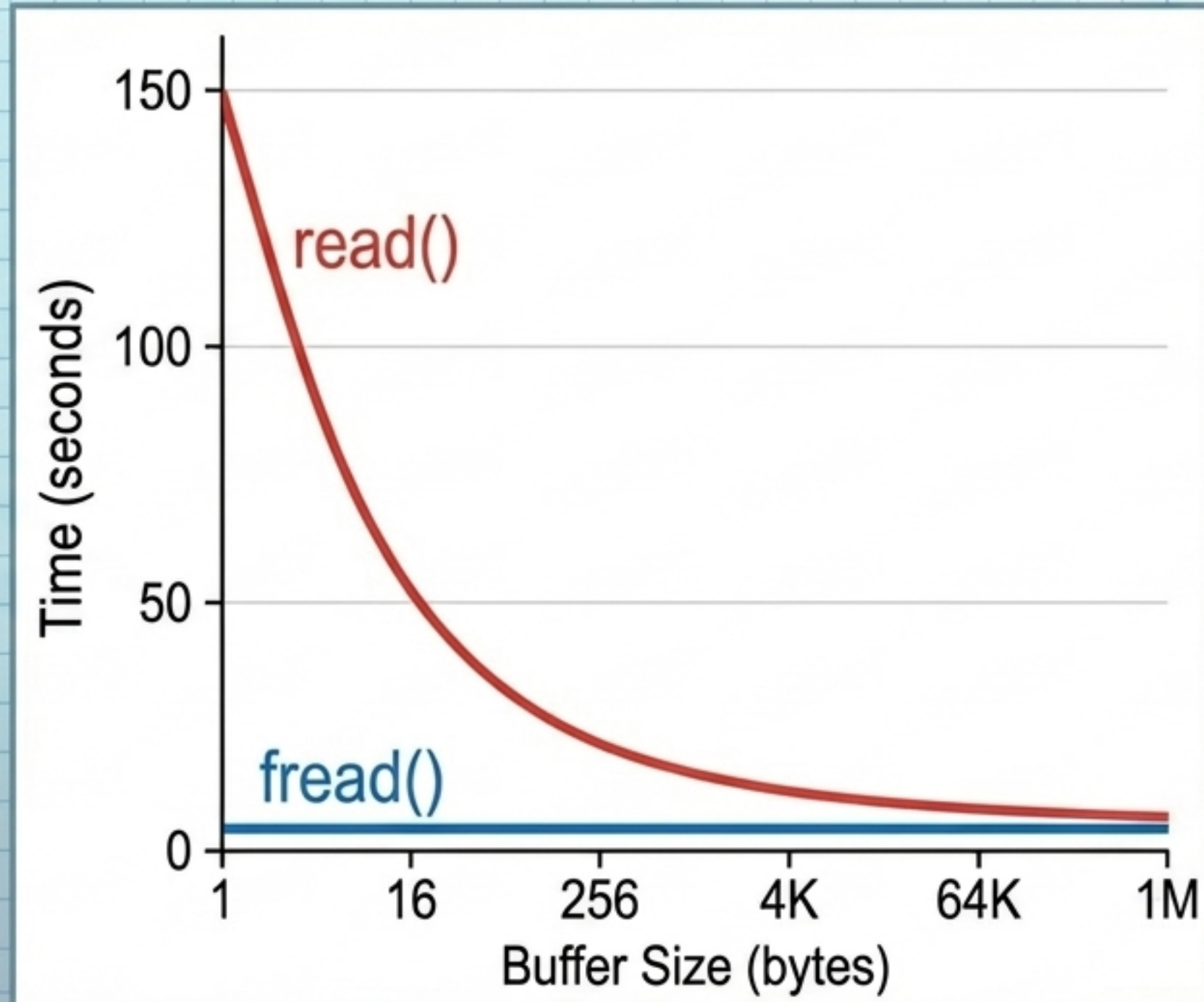


2.1s
Standart G/Ç
(Tamponlanmış)



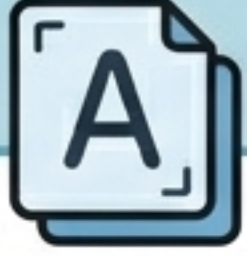
simpcat2 neden bu kadar yavaş? Çünkü 25 milyon baytlık bir dosya için 25 milyon kez işletim sistemine gidilir! Her bayt okumasında OS'in CPU'yu devralması, durumu kaydetmesi ve geri dönmesi devasa bir zaman kaybıdır.

Tamponlama (Buffering) Neden Hayat Kurtarır?



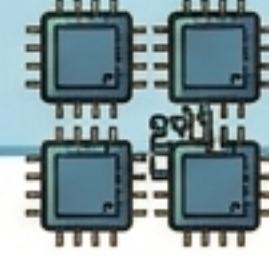
- **Mekanizma:** `getchar()` veya `fread()` çağrıldığında, kütüphane arka planda diske büyük bir `read()` işlemi yapar ve veriyi RAM'deki bir ara belleğe (tampona) alır. Sonraki çağrılar diske gitmeden anında RAM'den gelir.
- **Altın Kural:** Arabelleğe alma (**buffering**), pahalı sistem çağrılarının sayısını azaltmanın **en Unix'teki en etkili yoludur**.

Okuma İşlemleri: Doğru Fonksiyonu Seçme Rehberi



Tek Karakter İşlemleri

getchar() veya **fgetc()** kullanın. Sadece bir karakter okumak için son derece optimize edilmiştir.



Küçük Bayt Parçaları

fread() kullanın. C kütüphanesi sizin için arka planda tamponlama işlemlerini otomatik halleder.



Büyük Veri Yığınları

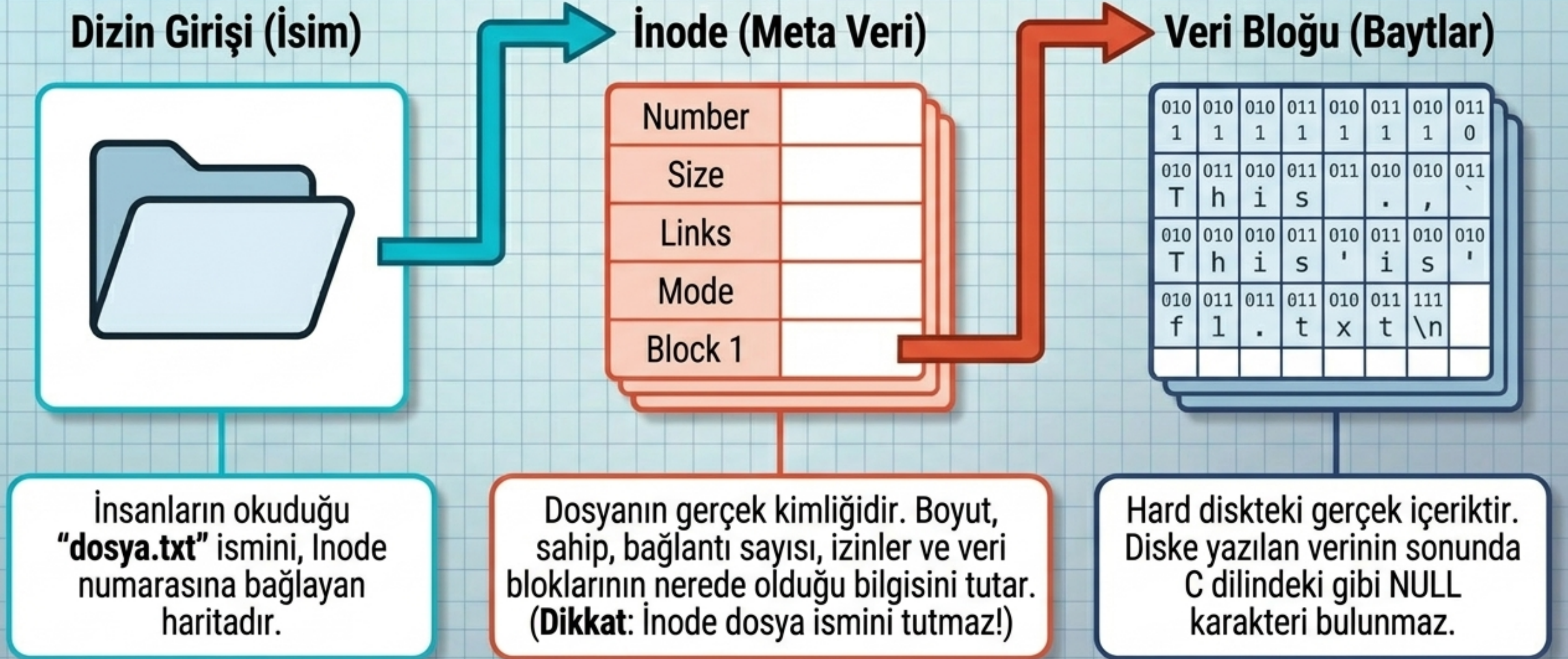
fread() ve **read()** benzer performans gösterir. Ancak hata denetimi ve tutarlılık için **fread()** tercih edilmelidir.



Kritik Sınav Detayı (Tuzak!)

getchar() bir char DEĞİL, bir **int** döndürür! Dosya sonu **EOF** değerini (-1) yakalayabilmek için geri dönüş tipi tamsayı olmak zorundadır.

Bir Dosyanın Anatomisi (Dosya Sadece İsim Değildir)



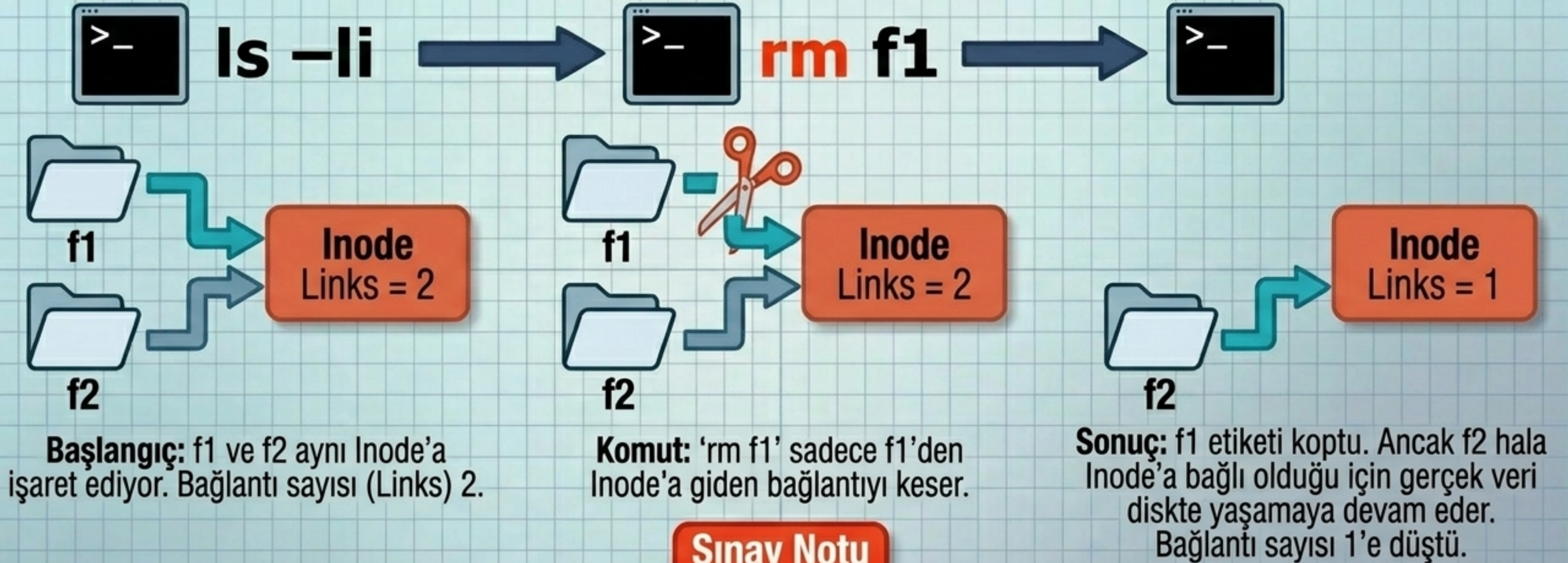
Hard Link Mekanizması Nasıl Çalışır?

`ln f1 f2`



- **İşlem:** Dosyayı KOPYALAMAZ. Sadece aynı Inode numarasına işaret eden yeni bir isim (f2) oluşturur.
- **Sonuç:** İkisi de aynı meta veriyi ve veri bloğunu paylaşır.
- **Etki:** f2'yi düzenlerseniz f1 de değişir. chmod ile f1'in izinlerini değiştirirseniz, f2 de etkilenir.
- **Meta Veri:** Inode içindeki 'Bağlantı Sayısı' (Links) 2 olarak güncellenir.

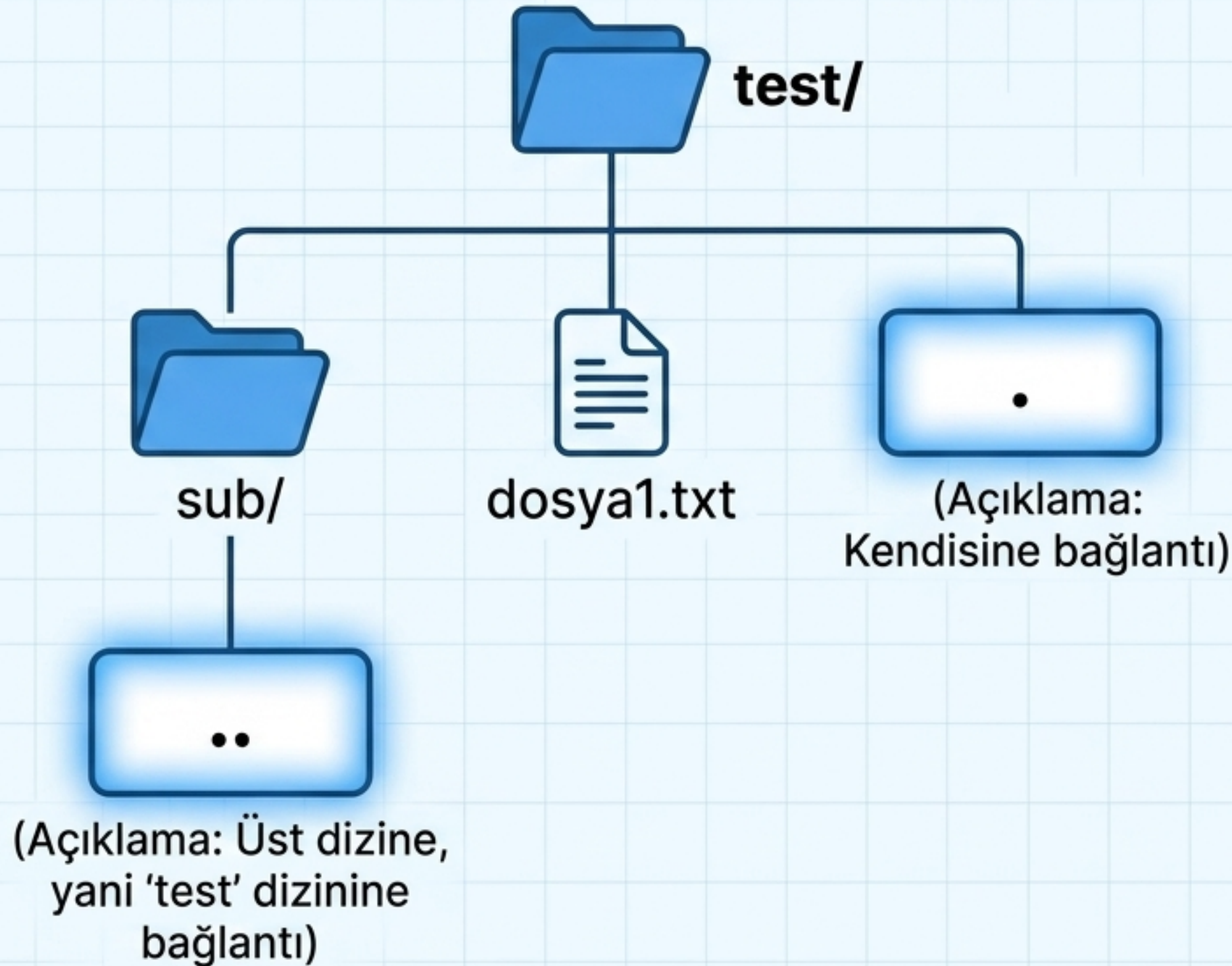
Silme işlemi (rm) Gerçekte Ne Yapar?



Sınav Notu

Bir dosyanın verisi ve Inode'u, sadece kendisine işaret eden **SON BAĞLANTI** kaldırıldığında (Link Count = 0 olduğunda) diskten tamamen silinir.

Dizinlerin (Klasörlerin) Gizli Bağlantıları



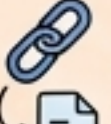
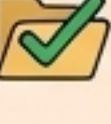
Soru: Yeni oluşturulan **boş bir dizinin neden en az 2 bağlantısı vardır?**

Cevap: Çünkü her dizin, kendi ismine ek olarak şunları içerir:

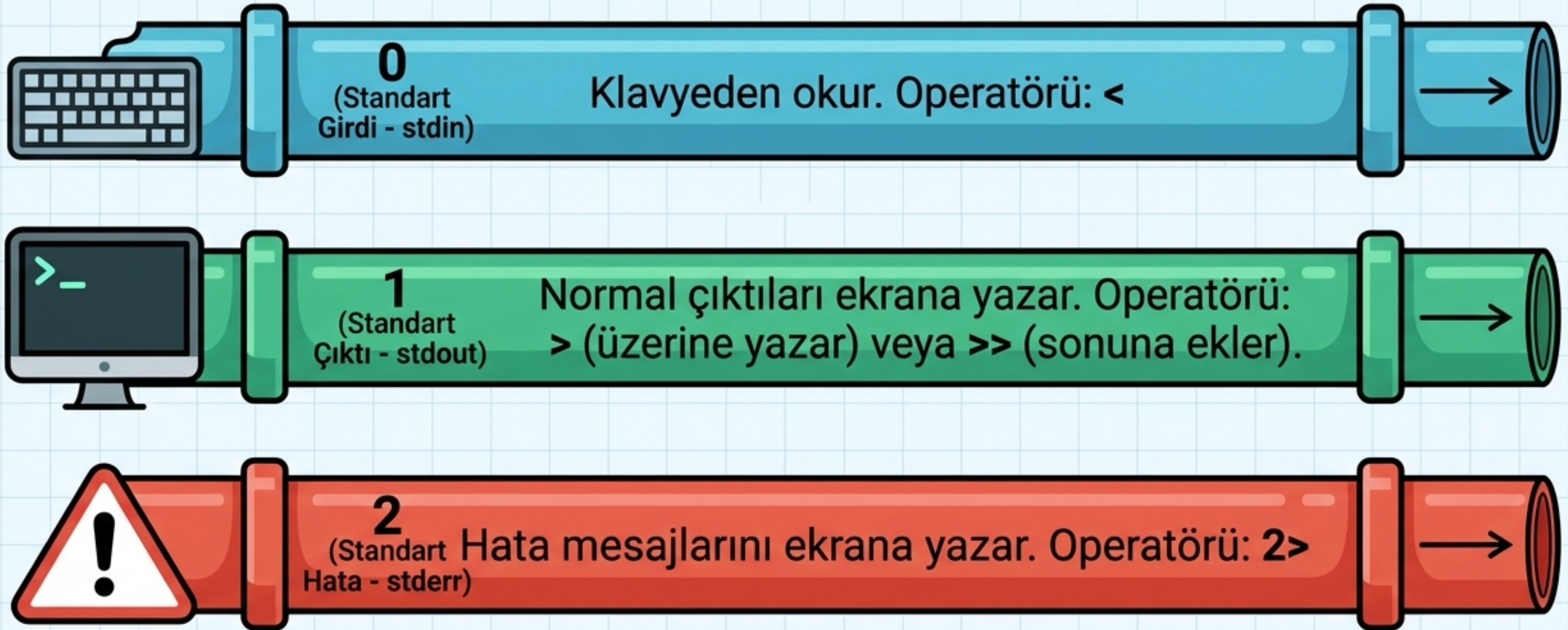
- **.** (**Nokta**): Dizin içinden kendisine olan gizli bağlantı.
- **..** (**İki Nokta**): Alt dizinlerden üst dizine (parent) olan bağlantı.

Yani 'test' içine 'sub' açarsanız, **'test/sub/..' sebebiyle 'test' dizininin bağlantı sayısı 3'e çıkar!**

Karşılaştırma: Hard Link vs. Soft (Sembolik) Link

<code>ln f1 f2</code>	<code>ln -s f1 f2</code>
Hard Link	Soft / Sembolik Link
 Inode: Aynı Inode'u paylaşırlar.  Dayanıklılık: Orijinal dosya (f1) silinirse, kopya (f2) sorunsuz çalışmaya devam eder (veri kaybolmaz).  Kısıtlama: Dizinlere (klasörlere) hard link yapılamaz. Bu sistem güvenliği için işletim sistemi tarafından kısıtlanmıştır.	 Inode: Tamamen farklı bir Inode'a sahiptir. Gerçek veriye değil, sadece orijinal dosyanın ismine (yoluna) işaret eden bir metin kısayoludur.  Dayanıklılık: Orijinal dosya silinirse veya taşınırsa, Soft Link KIRILIR ('dangling link' / unresolved) ve kullanılamaz hale gelir.  Esneklik: Dizinlere (klasörlere) sembolik bağlantı yapılabilir.

Kabuk Yönlendirmesine Giriş (Shell Redirection)



Çalışma Mantığı: Kabuk (Shell), yazdığınız komutları çalıştırmadan **ÖNCE** bu boruları (tanımlayıcıları) yönlendirme operatörlerine bakarak ilgili dosyalara bağlar.

Yönlendirme Sınav Tuzağı: Dosya Neden Boşalır?

Problem: Bu komut neden f2 dosyasının içeriğini tamamen boşaltır?

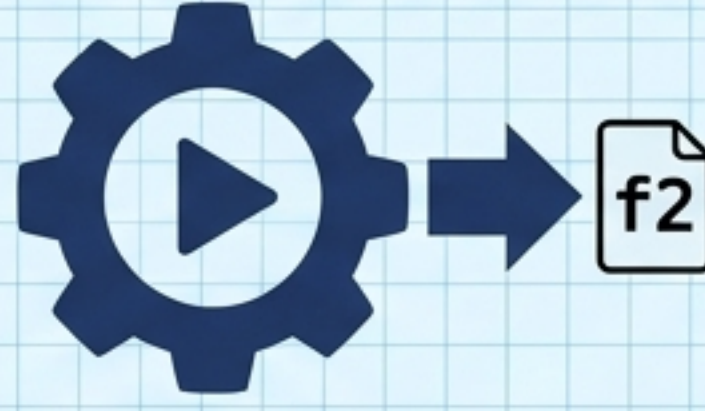
```
head < f2 > f2
```

Adım 1
Komut İşlenmeden Önce
(Kabuk Devrede)



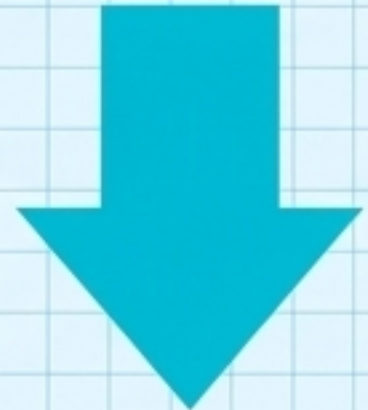
Kabuk, sağdaki '> f2' yönlendirmesini görür. f2 dosyası yazmak için açılır ve anında kesilip sıfırlanır (truncate edilir).

Adım 2
Komut Başlatılır



head programı çalıştırılır ve girdi olarak soldaki '< f2' üzerinden okuma yapmaya çalışır.

Adım 3
Kaçınılmaz Son



Ancak f2, Adım 1'de kabuk tarafından çoktan boşaltıldığı için okunacak hiçbir bayt kalmamıştır. Sonuç: Dosya sonsuza dek boş kalır.

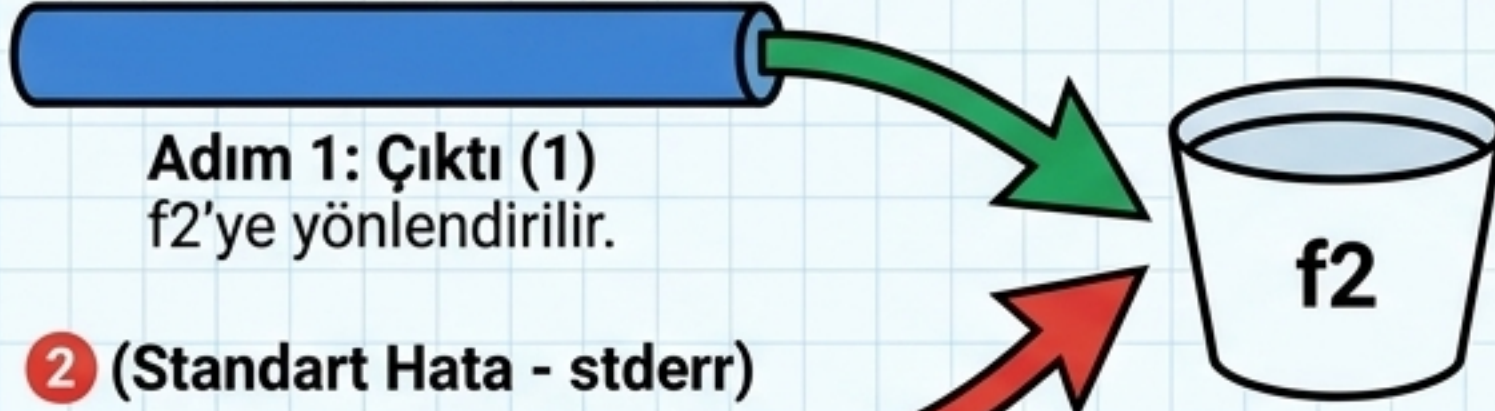
Hata ve Çıktıyı Birleştirme (2>&1) - Sıralamanın Önemi

Altın Kural: Kabuk, yönlendirmeleri daima **SOLDAN SAĞA** işler.

Doğru Kullanım

```
cat f1 f3 > f2 2>&1
```

1 (Standart Çıktı - stdout)



2 (Standart Hata - stderr)



Sonuç: Hem Çıktı hem de Hata f2 dosyasının içine başarıyla yazılır.

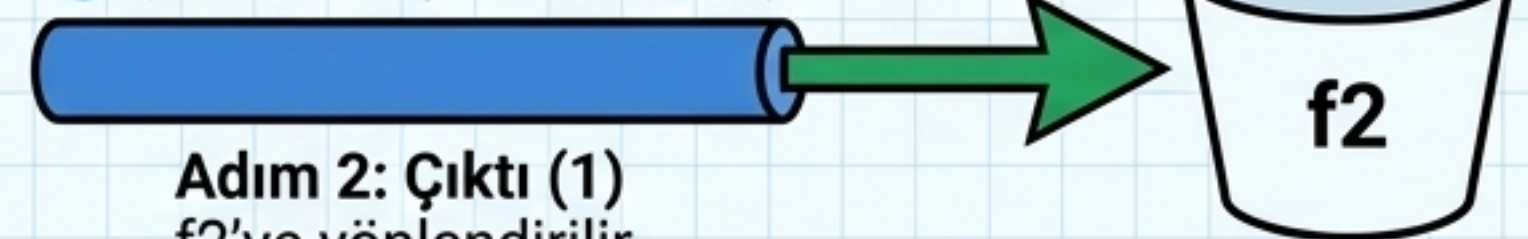
Klasik Yanıltmaca

```
cat f1 f3 2>&1 > f2
```

2 (Standart Hata - stderr)



1 (Standart Çıktı - stdout)



Sonuç: Hata Ekranda kalır! Sadece Çıktı f2'ye gider. (2, 1'in eski yerine bağlandı).

Sınav Öncesi Altın Sentez (Cheat Sheet)

1. Performans & G/Ç

Döngülerde tekil **read/write** (Sistem Çağrısı) kullanmaktan **kesinlikle** kaçının. C Standart Kütüphanesi (**fread/fwrite**) kullanarak otomatik tamponlamanın hızından faydalanın.

2. Değişken Tipleri

getchar() fonksiyonunun dönüş değerini daima **int** tipinde bir değişkene atayın (**char** kullanmayın). Aksi takdirde -1 olan **EOF** (Dosya Sonu) değerini doğru yakalayamazsınız.

3. Silme ve Link Mantığı

Dosya ismi sadece bir etikettir (dizin girişi). **rm** komutu yalnızca bu etiketi siler. Gerçek veri (İnode ve Disk Blokları) ancak Link Sayısı (bağlantı sayısı) sıfıra düştüğünde sistemden tamamen silinir.

4. Yönlendirme Sırası

Kabuk, operatörleri daima **soldan sağa** doğru işler ve asıl komutu başlatmadan önce çıktı dosyalarını açar/sıfırlar. Yönlendirmeleri sırasıyla zihninizde borular bağlar gibi canlandırın.